

Linux Embedded Systems

Mini Project Report

Student	Sylvan Arnold
Program	Computer Science
Academic year	2026
Class	MA_CSEL1
Repository	https://github.com/SylvanArnold/csel-workspace

1. Introduction

I implemented a complete fan control solution for a Linux embedded system.

The project consists of:

- An installable kernel module that controls the fan.
- A user-space daemon that displays fan data on a screen, listens for button inputs to configure the fan module, and exposes a Unix socket.
- A user-space application that connects to the daemon through the Unix socket to read data and configure the fan module.

Github repository: <https://github.com/SylvanArnold/csel-workspace>

Project structure:

- **src/07_miniproj/fan_driver**: Kernel module for fan control
- **src/07_miniproj/daemon**: User-space daemon for screen and LED control
- **src/07_miniproj/application**: User-space application for interacting with the fan controller
- **src/07_miniproj/common/fan_socket.h**: Socket communication definitions shared between the daemon and the application

2. Kernel Module

I created an installable kernel module using `module_platform_driver`, which can be loaded with `modprobe`. When loaded, the module requests the `fan-ctrl` pin from the device tree. After loading the module once, it is automatically loaded during every system boot and connected to the configured pin.

The following node must be added to the device tree:

```
fan-controller {
    compatible = "vendor,fan-controller";
    fan-gpios = <&pio 0 10 GPIO_ACTIVE_HIGH>;
    status = "okay";
};
```

Since no real fan is available, the module is connected to the board's status LED pin.

I created three sysfs entries: `manual_mode`, `frequency`, and `temperature`.

- `manual_mode` allows switching between manual and automatic control modes.
- `frequency` allows the user to set the fan frequency in manual mode and read the current frequency in automatic mode.
- `temperature` provides access to the current CPU temperature.

In automatic mode, the fan frequency is adjusted according to the CPU temperature, which is read from the `cpu-thermal` thermal zone. As requested in the project requirements, I implemented a simple lookup table to map temperature ranges to fan frequencies.

3. Daemon

The daemon uses the screen and the power LEDs. To enable I²C communication for screen control, the following node must be added to the device tree:

```
&i2c0 {  
    status = "okay";  
};
```

The daemon is implemented using an event-driven architecture based on `epoll` for event multiplexing. The entire application runs in a single thread. All events are handled by the `process_event` function.

The following events are handled:

- Button presses and releases
- Screen refresh timer expirations
- Unix socket events

When a button is pressed, the power LED is turned on; when the button is released, the LED is turned off.

When the screen refresh timer expires, the display is updated with the current fan data obtained from the kernel module.

When a client connects to the Unix socket, the daemon accepts the connection and adds the client file descriptor to the `epoll` instance. When a request is received, the daemon processes it, sends a response, and closes the connection.

At startup, the `daemonize` function is called to detach the process and run it in the background as a daemon.

4. User-Space Application

The user-space application connects to the daemon through the Unix socket and sends requests to read data or configure the fan module.

The application is intentionally simple and serves primarily as a demonstration of how external programs can interact with the daemon.

5. Conclusion

5.1. Summary

The solution worked as expected. Implementing the entire system in a single thread was challenging, but it was satisfying to discover how effectively `epoll` can be used for unified event handling in Linux.

Another challenge was selecting the appropriate Linux APIs on the module development part, as there are often multiple valid ways to implement the same functionality.

Overall, this project provided an excellent opportunity to apply the main concepts covered during the semester. It was a great preparation for the final exam.

5.2. Improvements

If I had more time, I would integrate the solution into my Buildroot configuration so that it would be included directly in the system image. This would allow the kernel module and the daemon to be installed and started automatically at boot, without any manual steps.